

PROJETO EDUCA+

Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas

INTEGRANTES DO PROJETO e RA'S

Bruno Sales	26028881
Gabriel Barbosa	23011348
Mel Oliva	26029351
Rayna Guimarães	26029441

São Paulo

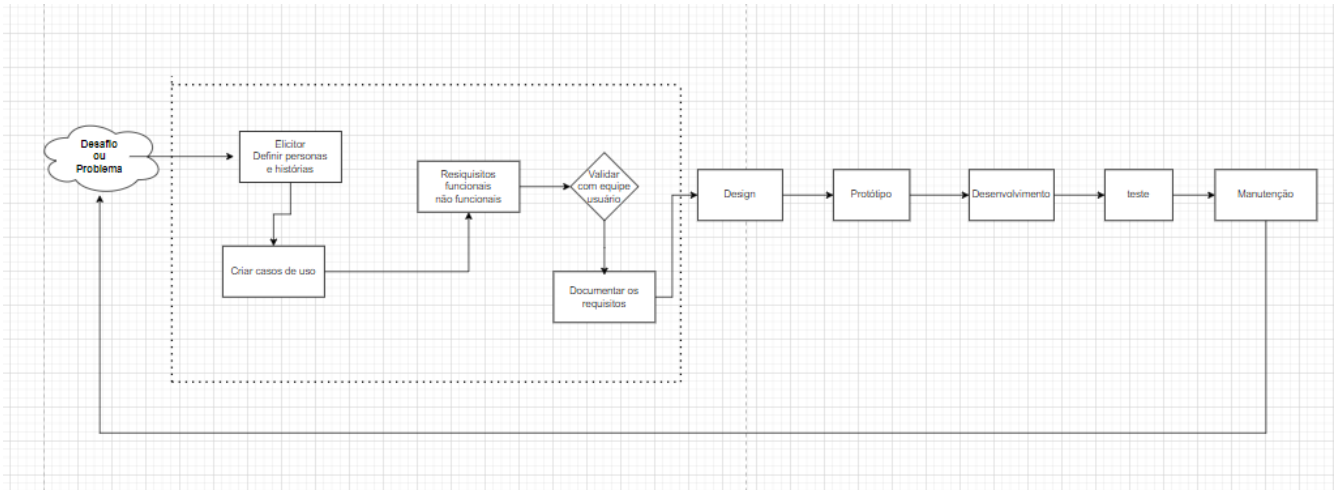
2025

1 INTRODUÇÃO	3
1.1 – Processo de desenvolvimento de Software	3
1.2 Apresentação	3
2. DESIGN SPRINT – Ideação e prototipação do desafio	4
2.1 Desafio	4
2.2 Entender Mapear	4
2.2.1 – Entendimento e observação. COMO PODEMOS?	4
2.2.2 – Pesquisa Desk	5
2.3 Ideação – desenho da solução (trilha do usuário)	7
2.4 Prototipagem	7
3. REQUISITOS DE SISTEMA	8
3.1 REQUISITOS FUNCIONAIS DE SOFTWARE	8
3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE	10
4. DIAGRAMAS DE UML	12
4.1 CASOS DE USO	13
4.2 DIAGRAMA DE CLASSE	14
5. Especificação estrutura de uma rotina do sistema	15
5.1 – Apresentar a especificação de acordo com o modelo abaixo	15
5.2 – Apresentar a codificação da rotina	15
6. ARQUITETURA DO SISTEMA	21
6.1. Visão Geral do Sistema	21
6.2. Arquitetura Geral	21
6.3. Componentes Arquiteturais e Camadas	21
6.3.1. Camada de Apresentação (Frontend)	21
6.3.2. Camada de Aplicação (Backend/API)	22
6.3.3. Camada de Domínio (Negócio)	22
6.3.4. Camada de Persistência (Banco de Dados)	22
6.4. Diagrama da Arquitetura	22
6.5. Organização de Pastas e Módulos	23
6.6. Integrações Externas (se houver)	23
6.7. Ambiente e Implantação	23
8. REFERÊNCIAS BIBLIOGRÁFICAS	23

1 INTRODUÇÃO

1.1 – Processo de desenvolvimento de Software

Exemplo (substituir) - o que foi realizado em sala no Draw.io, escolha um por equipe



1.2 Apresentação

A Messier Data & Creative é uma empresa de desenvolvimento de jogos e experiências interativas que combina tecnologia e criatividade para promover engajamento em eventos, demonstrações de produto e campanhas de marca. Como novo passo, a Messier pretende lançar uma plataforma de jogos digitais para escolas de ensino fundamental e médio. Neste Projeto Interdisciplinar, as equipes irão desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo. Fluxo do processo (simulação do módulo Escola): 1) Login: a escola se autentica; o aplicativo verifica se a escola possui pacote ativo e se o IP de origem (informado na simulação) está autorizado. 2) Catálogo: o aplicativo apresenta os games disponíveis no pacote adquirido pela escola. 3) Acesso: a escola simula o acesso a um game; o aplicativo registra e contabiliza o acesso no consumo do mês, respeitando o limite do pacote.

2. DESIGN SPRINT – Ideação e prototipação do desafio

2.1 Desafio

Inicial da proposta do PI - as equipes irão desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo.

Nosso projeto se inicia com a ideia de facilitar a vida das escolas no uso de jogos educacionais. Em vez de lidar com controles complicados de assinaturas, acessos e limites, o sistema propõe uma forma mais organizada, simples e segura de gerenciar tudo isso em um só lugar.

A intenção é ajudar as escolas a realmente entenderem como esses jogos estão sendo utilizados no dia a dia, acompanhando o progresso dos alunos e apoiando professores no uso da aplicação em sala de aula. Assim, o sistema passa a ser um apoio de ensino.

Portanto o nosso diferencial é pensar no futuro e transformar esse sistema em algo cada vez mais inteligente, que possa sugerir conteúdos, ajudar a identificar dificuldades dos alunos e tornar o aprendizado mais eficiente e interessante. No fim, a ideia é usar a tecnologia de forma prática e acessível para melhorar a experiência tanto de quem ensina quanto de quem aprende.

2.2 Entender Mapear

2.2.1 – Entendimento e observação. COMO PODEMOS?

Como podemos transformar o uso excessivo de telas em uma ferramenta poderosa para a educação? A resposta pode estar no desenvolvimento de um software de jogos educativos.

A ideia central é ajudar as escolas a incentivar a educação por meio da tecnologia, aproveitando um cenário que já é realidade desde a pandemia, quando o tempo de exposição às telas aumentou significativamente. Em vez de combater esse hábito, o software propõe ressignificá-lo, transformando o tempo digital em uma experiência produtiva e enriquecedora.

É essencial que o sistema seja acessível, permitindo que os professores e as escolas o utilizem sem dificuldades. Interfaces simples, comandos claros e suporte integrado são fundamentais para garantir que o foco não esteja apenas na tecnologia, mas também no aprendizado. Quanto mais fácil for para o educador aplicar a ferramenta, maior será sua adesão e impacto em sala de aula.

Ao mesmo tempo, o software deve estimular as crianças por meio de jogos interativos e educacionais, que combinem diversão e conteúdo pedagógico. Elementos como vídeos interativos, desafios progressivos e recompensas visuais ajudam a prender a atenção, criando uma experiência imersiva que compete diretamente com outras formas de entretenimento digital. Outro ponto importante é o uso da competitividade de forma positiva, com linguagem contemporânea e próxima da realidade dos

alunos. Expressões atuais, podem ser incorporadas para tornar o ambiente mais familiar e motivador, incentivando a participação e o engajamento entre os estudantes.

Assim, ao integrar tecnologia, educação e cultura digital, o software não apenas acompanha as mudanças do mundo moderno, mas também serve como uma solução prática para um dos maiores desafios atuais: como educar de forma eficaz em um ambiente cada vez mais digital.

2.2.2 – Pesquisa Desk

As consequências do uso excessivo e precoce de dispositivos eletrônicos no processo de desenvolvimento das novas gerações: A exposição intensa às telas afeta diretamente o funcionamento do cérebro de crianças e adolescentes. Especialmente nos primeiros anos de vida, o desenvolvimento neural dos seres humanos é moldado pela interação com o ambiente físico e o convívio com as pessoas ao seu redor, o que não acontece nos espaços digitais.

Correr, pular, brincar com outras crianças, manipular objetos e conversar hábitos que ativam múltiplas áreas do cérebro, contribuindo para o amadurecimento neurológico. Quando a criança passa horas apenas consumindo estímulos audiovisuais, o cérebro não é desafiado a desenvolver funções complexas, como a empatia, o autocontrole, a criatividade ou a linguagem.

O conteúdo encontrado nas redes sociais e nos aplicativos voltados para o público infantil costuma ser marcado por estímulos rápidos, coloridos e sonoros. Esses elementos ativam os circuitos de recompensa do cérebro de forma intensa, o que torna a experiência viciante e reduz a capacidade de concentração em atividades mais lentas e complexas, como a leitura ou o estudo.

Benefícios dos jogos educativos: Tanto físico como digitais, os benefícios dos jogos na educação podem ser variados: Oportunizam o uso de novas abordagens dos conteúdos, de forma lúdica e divertida;

Incentivam o engajamento nas atividades de sala de aula e da escola;

Promovem situações-problema e atividades desafiadoras, que podem se relacionar às diversas áreas do conhecimento;

Criam condições para desenvolvimento do raciocínio lógico, para o desenvolvimento cognitivo, emocional e também motor;

Fortalecem trocas entre estudantes e professores, criando um clima de integração e socialização;

Favorecem o trabalho com a interdisciplinaridade, a linguagem e a comunicação na escola.

De acordo com a etapa escolar, o jogo, situado no campo da brincadeira, pode ser ainda mais significativo. Os jogos na educação infantil, por exemplo, podem ser importantes para o desenvolvimento cognitivo e motor das crianças.

Existem diversas pesquisas científicas que exploram o uso de jogos na educação e seus benefícios. De acordo com o artigo *Uso de jogos didáticos no processo ensino-aprendizagem*, os benefícios dos jogos didáticos incluem o estímulo à motivação, desenvolvimento de habilidades e aprimoramento do

processo de ensino. O texto conclui enfatizando a importância dos jogos como ferramentas dinâmicas e eficazes no ensino-aprendizagem, proporcionando um ambiente harmonioso e divertido para o crescimento intelectual dos alunos e contribuindo para a formação de cidadãos críticos e reflexivos.

Já a pesquisa *Uso de jogos educacionais no processo de ensino e de aprendizagem* explora a relevância e a integração dos jogos, especialmente os digitais, na educação. Reconhecendo que jogos sempre foram parte essencial da vida humana, especialmente na infância, a pesquisa argumenta que os jogos digitais são ferramentas valiosas no processo educativo. Eles oferecem motivação e desenvolvem persistência em crianças em idade escolar, tornando o aprendizado mais divertido e relevante.

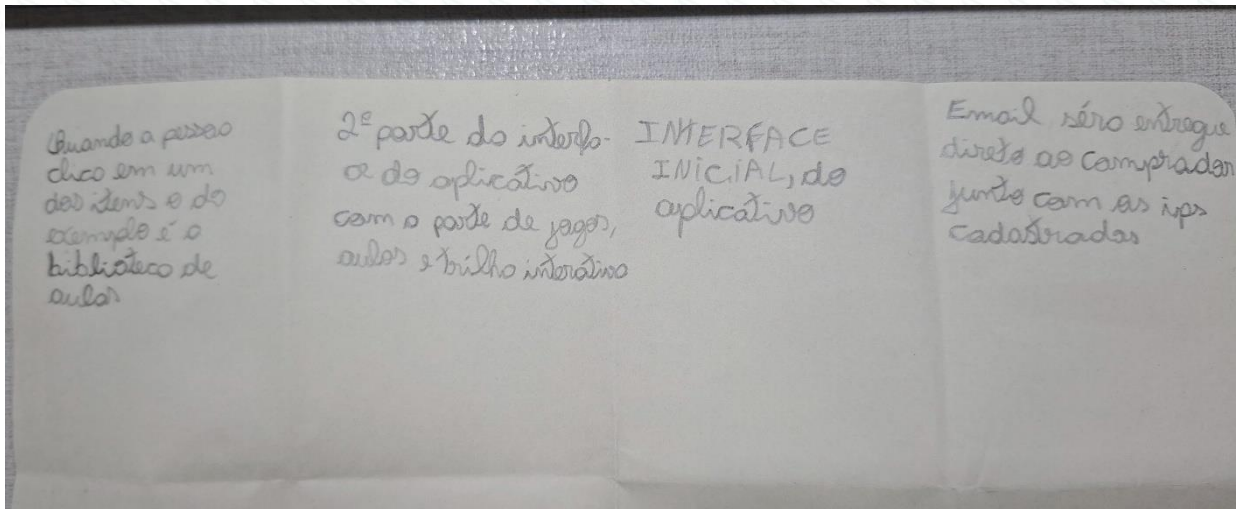
Rotina de estudos em casa ajuda no aprendizado das crianças: A rotina de estudos na infância tem impacto direto no desempenho escolar e no equilíbrio emocional. Segundo especialistas, esses hábitos formam bases emocionais e cognitivas que acompanham o estudante ao longo da vida e fazem diferença também nas fases mais exigentes da educação, como os vestibulares e o Exame Nacional do Ensino Médio (Enem), portas de entrada para uma formação em nível superior.

“A rotina ajuda a criança a entender limites, responsabilidades e a se organizar. Esses aprendizados vão muito além da escola, são valores que ela leva para o futuro. A constância e a organização desenvolvidas na infância ajudam a estruturar competências que seguem pela vida”, diz a professora Denise.

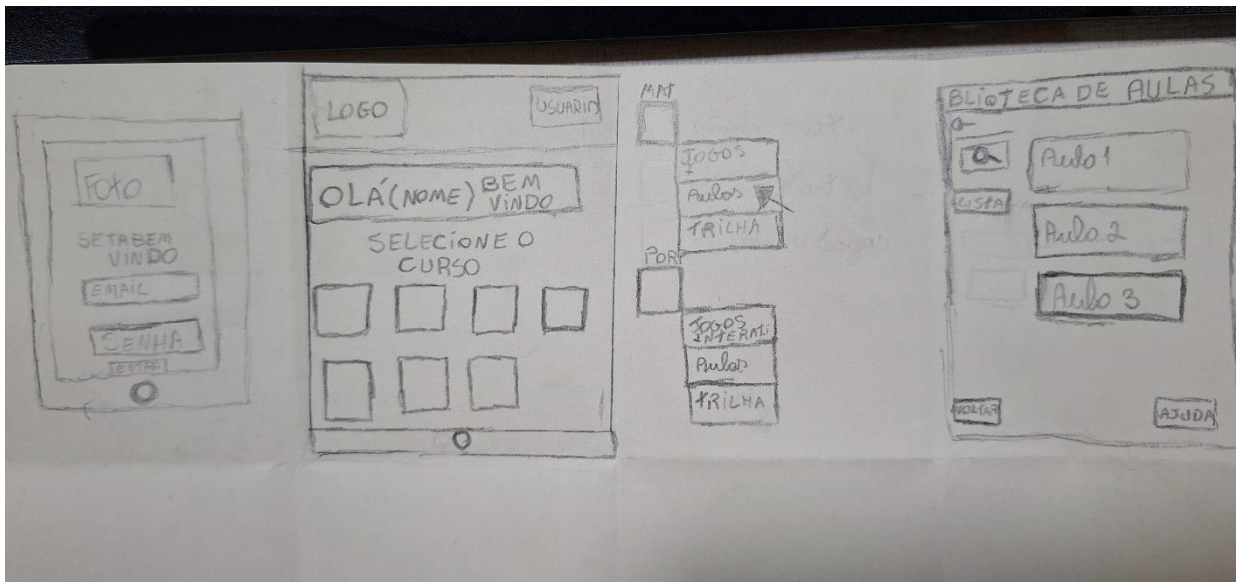
A psicóloga educacional Tatiana Rabitto reforça que a organização da rotina contribui para a estabilidade emocional e o fortalecimento da autoconfiança das crianças.

“Quando a criança tem rotina, ela se sente segura. Isso ajuda na concentração, na autoconfiança e no prazer em aprender. A forma como os pais lidam com o momento de estudo influencia muito. O ideal é transformar esse tempo em algo de afeto e presença, não em cobrança”, explica Rabitto.

2.3 Ideação – desenho da solução (trilha do usuário)

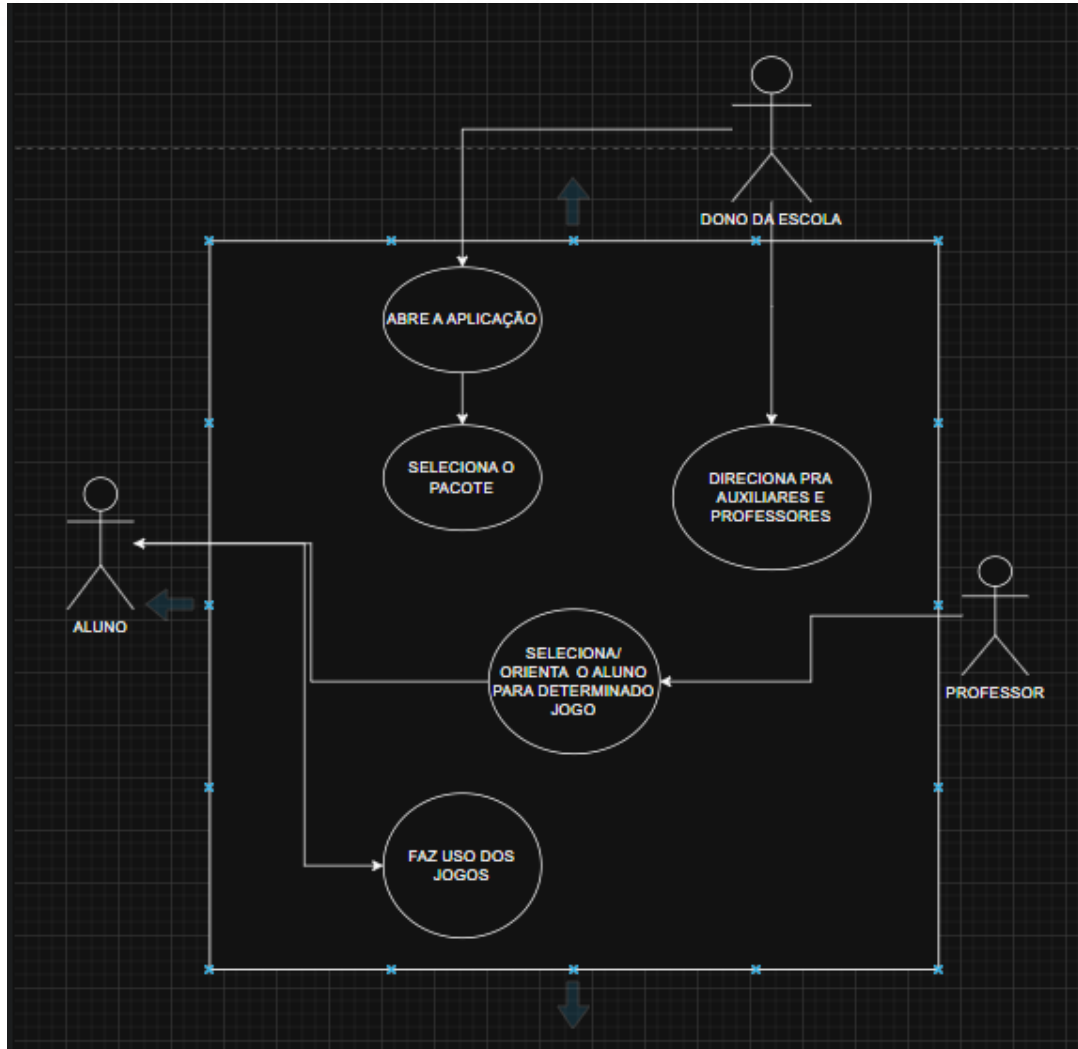


2.4 Prototipagem



3. REQUISITOS DE SISTEMA

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE



Necessários 6 requisitos

RFS01	
Função	Autenticação da Escola
Descrição	O sistema deve permitir que a escola realize login com e-mail e senha, verificando se o cadastro está ativo no banco de dados
Entradas	E-mail da escola, senha
Fonte	Escola (usuário do sistema)
Saídas	Mensagem de sucesso e redirecionamento ao menu principal, ou mensagem de erro de credenciais inválidas

Ação	Verificar e-mail e senha no banco de dados; se válidos, iniciar sessão autenticada da escola
-------------	--

RFS02	
Função	Validação de IP Autorizado
Descrição	O sistema deve verificar se o IP de origem informado no momento do login está cadastrado como autorizado para aquela escola
Entradas	IP de origem (informado na simulação de login), ID da escola
Fonte	Escola (usuário do sistema)
Saídas	Permissão de acesso ou mensagem de bloqueio "IP não autorizado"
Ação	Consultar a lista de IPs autorizados vinculados à escola; se o IP não estiver cadastrado, bloquear o acesso

RFS03	
Função	Verificação de Pacote Ativo
Descrição	O sistema deve verificar se a escola possui uma assinatura ativa e dentro da validade antes de liberar o acesso ao catálogo
Entradas	ID da escola, data atual
Fonte	Banco de dados de assinaturas
Saídas	Liberação do catálogo, ou mensagem "Pacote inativo ou vencido"
Ação	Consultar o pacote vinculado à escola; verificar status ("ATIVO") e data de validade; bloquear acesso caso não esteja regular

RFS04	
Função	Exibição do Catálogo de Jogos
Descrição	O sistema deve apresentar ao usuário apenas os jogos incluídos no pacote contratado pela escola, com nome, categoria e faixa etária
Entradas	ID da escola, ID do pacote contratado
Fonte	Banco de dados de jogos e pacotes
Saídas	Lista de jogos disponíveis filtrada pelo pacote

Ação	Buscar no catálogo os jogos vinculados ao pacote da escola e exibi-los; jogos de outros pacotes não devem aparecer
-------------	--

RFS05	
Função	Registro e Contabilização de Acesso a Jogo
Descrição	O sistema deve registrar cada acesso simulado a um jogo e contabilizá-lo no consumo mensal da escola, respeitando o limite do pacote
Entradas	ID da escola, ID do jogo acessado, data/hora, IP de origem
Fonte	Escola (ao selecionar um jogo no catálogo)
Saídas	Confirmação de acesso registrado, ou mensagem "Limite mensal de acessos atingido"
Ação	Verificar se o número de acessos do mês ainda é menor que o limite do pacote; se sim, registrar o acesso e incrementar o contador; se não, bloquear e exibir mensagem de limite atingido

RFS06	
Função	Geração de Relatório de Consumo
Descrição	O sistema deve gerar um relatório com o histórico de acessos realizados pela escola no mês, mostrando jogos acessados, datas e total de acessos consumidos
Entradas	ID da escola, mês de referência
Fonte	Administrador ou escola autenticada
Saídas	Relatório com lista de acessos do mês, total utilizado e saldo restante do pacote
Ação	Consultar todos os registros de acesso do mês para aquela escola e consolidar as informações em um relatório exibido em tela

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE

Necessários 6 requisitos

RFS01	
Função	Segurança — Criptografia de Senhas
Descrição	As senhas das escolas devem ser armazenadas com hash criptográfico (ex: bcrypt), nunca em texto puro
Entradas	Senha informada no cadastro ou login

Fonte	Escola (usuário)
Saídas	Senha armazenada de forma segura; autenticação realizada via comparação de hash
Ação	Aplicar algoritmo de hash na senha antes de salvar no banco; no login, comparar o hash gerado com o armazenado

RFS02	
Função	Desempenho — Tempo de Resposta
Descrição	O sistema deve responder a qualquer operação (login, carregamento do catálogo, registro de acesso) em no máximo 3 segundos em condições normais de uso
Entradas	Qualquer requisição do usuário
Fonte	Escola ou Administrador
Saídas	Resposta da operação dentro do tempo limite
Ação	Otimizar consultas ao banco de dados; utilizar índices nas tabelas de escola, pacote e registro de acesso para garantir agilidade nas buscas

RFS03	
Função	Usabilidade — Interface Intuitiva
Descrição	A interface do aplicativo desktop deve ser simples e intuitiva, de forma que usuários sem treinamento técnico (coordenadores e secretários de escola) consigam operá-la sem dificuldades
Entradas	Interações do usuário com a interface (cliques, formulários)
Fonte	Escola (usuário final)
Saídas	Navegação fluida com mensagens de feedback claras para cada ação
Ação	Projetar telas com poucos elementos por página, mensagens de erro descritivas e fluxo linear (login → catálogo → acesso)

RFS04	
Função	Confiabilidade — Disponibilidade do Sistema
Descrição	O sistema deve estar disponível durante o horário escolar (7h às 22h, dias úteis) com disponibilidade mínima de 99% nesse período

Entradas	Tentativa de acesso ao sistema por parte da escola
Fonte	Escola (usuário)
Saídas	Sistema disponível e funcional dentro do horário estabelecido
Ação	Implementar monitoramento de disponibilidade; em caso de falha, exibir mensagem clara ao usuário e registrar o incidente para análise

RFS05	
Função	Manutenibilidade — Código Modular
Descrição	O sistema deve ser desenvolvido de forma modular e com código documentado, facilitando manutenções futuras e a adição de novos jogos ou pacotes sem necessidade de reestruturar o sistema
Entradas	Alterações no código-fonte, novos módulos ou pacotes
Fonte	Equipe de desenvolvimento
Saídas	Sistema atualizado com mínimo de impacto nas funcionalidades existentes
Ação	Utilizar arquitetura em camadas (apresentação, negócio, dados); separar regras de negócio do código de interface; documentar funções e classes principais

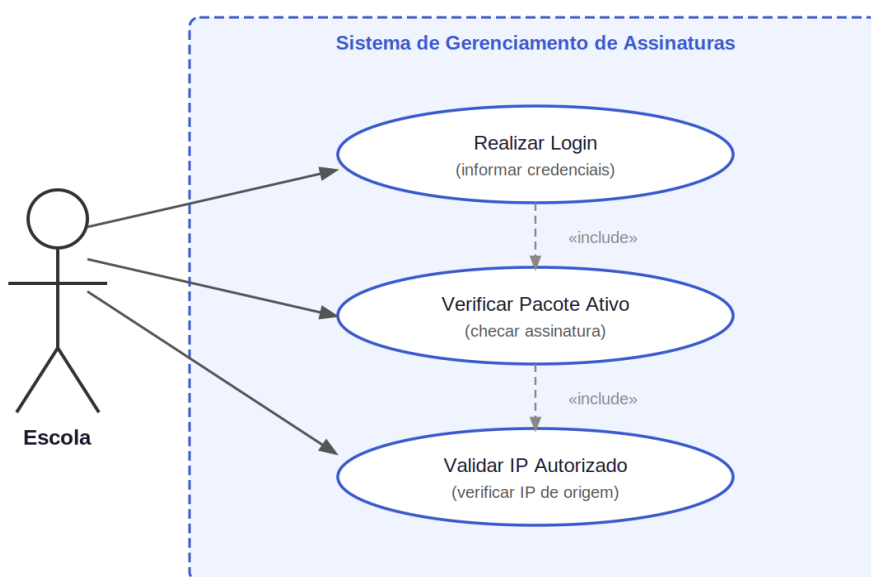
RFS06	
Função	Portabilidade — Compatibilidade com Windows
Descrição	O aplicativo desktop deve funcionar corretamente nas versões Windows 10 e Windows 11, que são os sistemas operacionais mais utilizados nas escolas brasileiras
Entradas	Instalação e execução do aplicativo no ambiente da escola
Fonte	Escola (ambiente de instalação)
Saídas	Sistema funcionando corretamente sem erros de compatibilidade
Ação	Desenvolver e testar o aplicativo nos dois sistemas operacionais; utilizar bibliotecas e frameworks com suporte estável para Windows

4. DIAGRAMAS DE UML

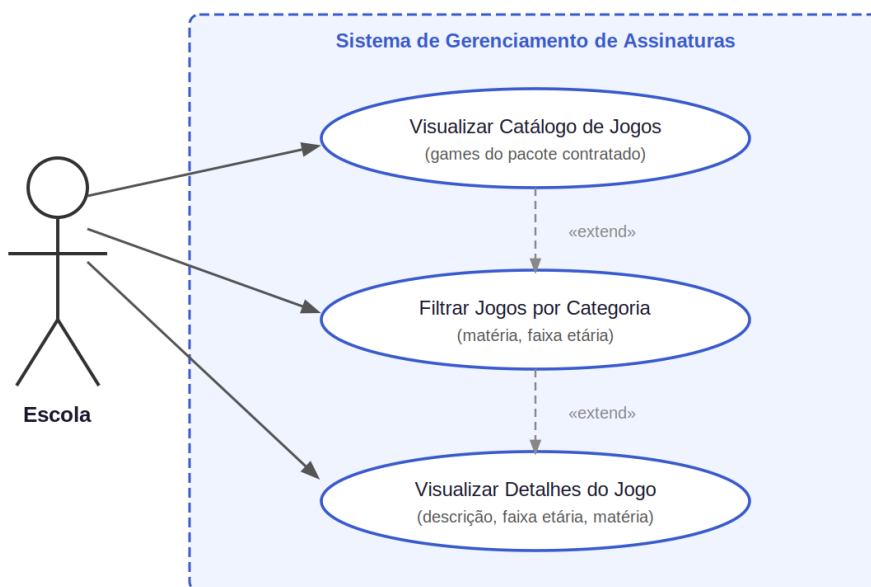
4.1 CASOS DE USO

Apresentar 3 casos de uso do sistema

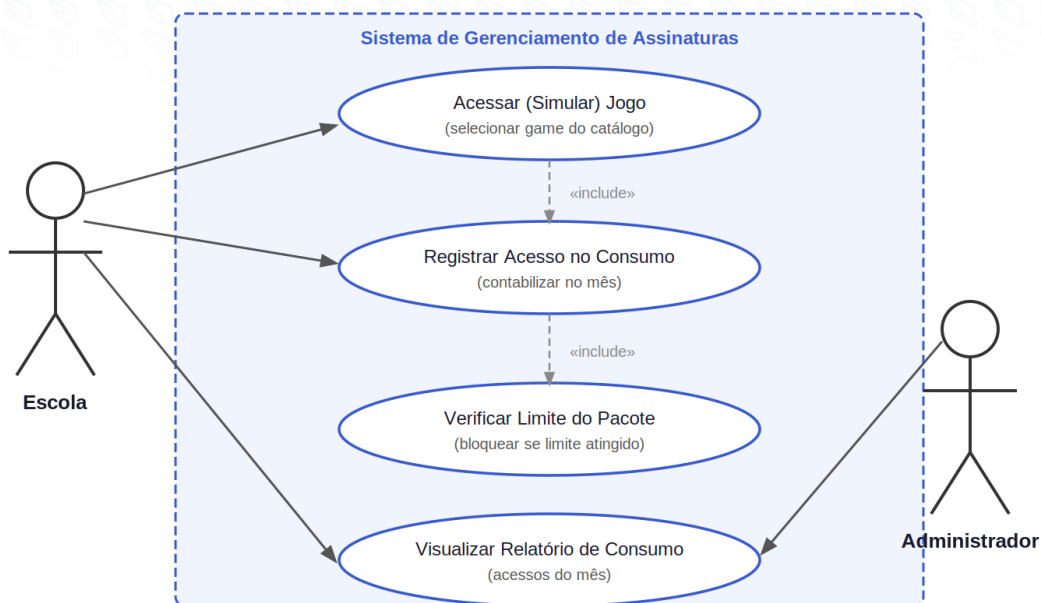
1º Caso: Login e Autenticação da Escola



2º Caso: Acesso ao Catálogo de Jogos



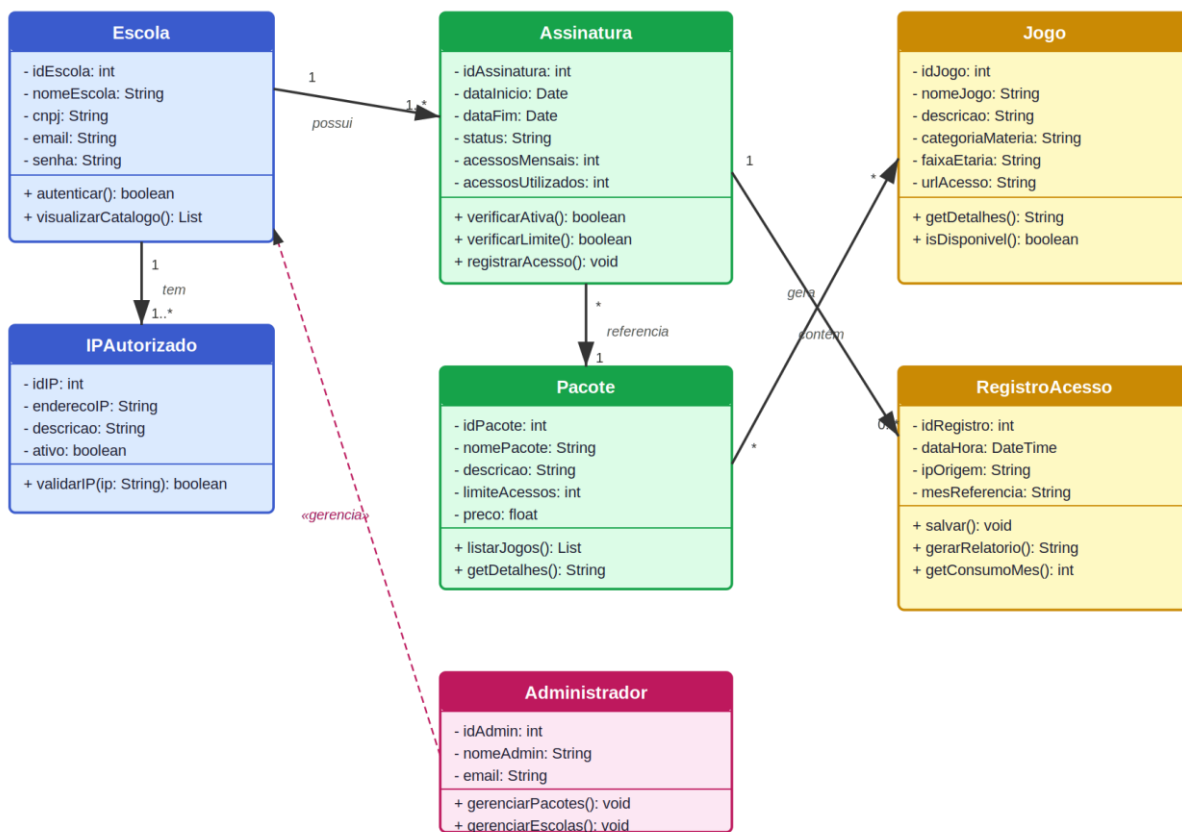
3º Caso: Acesso a Jogo e Controle de Consumo



4.2 DIAGRAMA DE CLASSE

Apresentar 1 diagrama de classe do sistema

Diagrama de Classes – Sistema de Assinaturas de Jogos Educacionais



5. Especificação estrutura de uma rotina do sistema

5.1 – Apresentar a especificação de acordo com o modelo abaixo

Sistema/Software

Função - Exibir os games disponíveis

Descrição - Apresenta os jogos disponíveis de acordo com o pacote contratado pela escola

Entradas - Id dos jogos, Id da escola, pacote da escola

Fonte – Lista de games

Saídas - Lista de jogos disponíveis + mensagem “jogo encontrado”

Destino - Página de acesso/login

Ação - Realizar a busca dos jogos no catálogo, verificar a validade do pacote da escola, identificar os jogos incluídos no pacote e permitir ou bloquear a exibição conforme as permissões

Requisitos - Os jogos exibidos devem estar incluídos no pacote contratado e o pacote deve estar ativo/regular

Pré-condição - Escola autenticada no sistema e jogos cadastrados no banco de dados

Pós-condições - O aluno consegue acessar o jogo

Efeitos colaterais - Atualizações constantes no sistema

5.2 – Apresentar a codificação da rotina

Apresente em pseudocódigo, Python ou outra linguagem

"""

Sistema de Exibicao de Games por Pacote Escolar

=====

"""

from datetime import date

#

=====

DADOS DO SISTEMA

#

=====

CATALOGO_GAMES = {

"G001": {"nome": "Matematica Aventura", "categoria": "Matematica"},

"G002": {"nome": "Palavras Magicas", "categoria": "Portugues"},

"G003": {"nome": "Ciencias Espaciais", "categoria": "Ciencias"},

"G004": {"nome": "Historia do Brasil", "categoria": "Historia"},

"G005": {"nome": "Ingles Divertido", "categoria": "Ingles"},

"G006": {"nome": "Geometria 3D", "categoria": "Matematica"},

"G007": {"nome": "Ecologia em Acao", "categoria": "Ciencias"},

"G008": {"nome": "Quiz Mundial", "categoria": "Geografia"},

}

PACOTES = {

"BASICO": {

"descricao": "Pacote Basico",

"games_incluidos": ["G001", "G002", "G005"],

},

"INTERMEDIARIO": {

"descricao": "Pacote Intermediario",

```

    "games_incluidos": ["G001", "G002", "G003", "G004", "G005"],
},
"PREMIUM": {
    "descricao": "Pacote Premium",
    "games_incluidos": ["G001", "G002", "G003", "G004", "G005", "G006", "G007",
"G008"],
},
}

ESCOLAS = {
    "ESC001": {"nome": "Escola Estadual Sao Paulo", "pacote": "BASICO",    "status":
"ATIVO", "validade": date(2027, 12, 31)},
    "ESC002": {"nome": "Colegio Instituto Federal", "pacote": "PREMIUM",    "status":
"ATIVO", "validade": date(2030, 6, 30)},
    "ESC003": {"nome": "Escola Municipal Centro", "pacote": "INTERMEDIARIO", "status":
"INATIVO", "validade": date(2026, 12, 31)},
}

#
=====
=====

# FUNCOES

#
=====
=====

def autenticar_escola(id_escola):
    escola = ESCOLAS.get(id_escola)
    if escola:

```



```
print("Escola autenticada: " + escola["nome"])
else:
    print("Escola nao encontrada: " + id_escola)
return escola

def verificar_pacote(escola):
    if escola["status"] != "ATIVO":
        print("Pacote inativo. Acesso bloqueado.")
        return False
    if escola["validade"] < date.today():
        print("Pacote vencido em " + str(escola["validade"]) + ". Acesso bloqueado.")
        return False
    return True

def buscar_games_disponiveis(id_escola):
    print("")
    print("Sistema de Games - Pagina de Acesso")
    print("-----")

    escola = autenticar_escola(id_escola)
    if not escola:
        return []

    pacote_id = escola["pacote"]
    pacote = PACOTES.get(pacote_id)
```

```
print("Pacote contratado: " + pacote["descricao"])
```

```
print("Status: " + escola["status"])
```

```
print("Validade: " + str(escola["validade"]))
```

```
if not verificar_pacote(escola):
```

```
    print("Nenhum jogo disponivel para esta escola.")
```

```
    return []
```

```
ids_do_pacote = pacote["games_incluidos"]
```

```
games_encontrados = []
```

```
print("")
```

```
print("Buscando jogos no catalogo...")
```

```
print("")
```

```
for id_game in ids_do_pacote:
```

```
    game = CATALOGO_GAMES.get(id_game)
```

```
    if game:
```

```
        games_encontrados.append({
```

```
            "id": id_game,
```

```
            "nome": game["nome"],
```

```
            "categoria": game["categoria"],
```

```
        })
```

```
if games_encontrados:
```

```
    print(str(len(games_encontrados)) + " jogo(s) disponivel(is):")
```

```
print("")

for i, g in enumerate(games_encontrados, 1):

    print(str(i) + ". " + g["id"] + " - " + g["nome"] + " | " + g["categoria"])

print("")

print("Mensagem do sistema: jogo encontrado")

else:

    print("Nenhum jogo disponivel no momento.")


print("-----")

return games_encontrados


#
=====

# EXECUCAO

#
=====

if __name__ == "__main__":

    nome_digitado = input("Digite o nome da escola: ")

    escola_encontrada = None

    id_encontrado = None

    for id_escola, dados in ESCOLAS.items():

        if dados["nome"].lower() == nome_digitado.lower():
```



```
escola_encontrada = dados
```

```
id_encontrado = id_escola
```

```
break
```

```
if not escola_encontrada:
```

```
    print("Escola nao encontrada. Verifique o nome e tente novamente.")
```

```
else:
```

```
    buscar_games_disponiveis(id_encontrado)
```

6. ARQUITETURA DO SISTEMA

Para a arquitetura do sistema, considere o template abaixo:

Template (esquema de arquitetura) para orientar a elaboração de um **Projeto de Arquitetura de Sistema Web**, cobrindo desde a camada de apresentação até a persistência de dados, com foco em modularidade, escalabilidade e eficiência.

6.1. Visão Geral do Sistema

- **Nome do Sistema:**
- **Objetivo do Sistema:** (Breve descrição da finalidade do sistema)
- **Público-Alvo:**
- **Principais Funcionalidades:**

6.2. Arquitetura Geral

Descrever o estilo arquitetural adotado (por exemplo, MVC, camadas, microsserviços, hexagonal, etc.):

Exemplo: Arquitetura em Camadas (Layered Architecture) com separação clara entre apresentação, lógica de negócios e persistência.

6.3. Componentes Arquiteturais e Camadas

6.3.1. Camada de Apresentação (Frontend)

- **Tecnologias:** (Ex.: HTML5, CSS3, JavaScript, React/Vue/Angular)
- **Responsabilidades:**
 - o Interface com o usuário
 - o Validação inicial de dados

- o Comunicação com a API via HTTP/JSON

6.3.2. Camada de Aplicação (Backend/API)

- **Tecnologias:** (Ex.: Node.js, Django, Spring Boot, Flask, Laravel)
- **Responsabilidades:**
 - o Implementação da lógica de negócio
 - o Autenticação/autorização
 - o Comunicação com a camada de dados
 - o Exposição de endpoints RESTful/GraphQL

6.3.3. Camada de Domínio (Negócio)

- **Descrição:**
 - o Contém as regras de negócio e os modelos de domínio
 - o Independente das demais camadas
- **Componentes:**
 - o Entidades
 - o Casos de uso
 - o Regras de validação

6.3.4. Camada de Persistência (Banco de Dados)

- **Tecnologias:** (Ex.: PostgreSQL, MySQL, MongoDB, SQLite)
- **Responsabilidades:**
 - o Armazenamento e recuperação de dados
 - o Scripts de criação e migração de banco

6.4. Diagrama da Arquitetura

Inserir um **diagrama estrutural** (ex.: em draw.io, Lucidchart ou outro) mostrando:

- Fluxo entre frontend, backend, banco de dados
- Componentes principais
- APIs e serviços envolvidos
- Gateways, autenticação, filas de mensagens (se houver)

Exemplo textual de uma arquitetura de sistema web.

```

[Usuário/Browser]
|
React (Frontend)
|
[HTTP/JSON]
|
Node.js + Express (API REST)
    
```

JavaScript ou SQL

PostgreSQL (Banco de Dados)

Diagrama:

6.5. Organização de Pastas e Módulos

Descrever como o código será organizado no repositório:

```

/ frontend      / components/    / pages/        / services/    / backend
/ controllers/  / services/      / models/       / routes/      / config/
/database      / schema.sql      / migrations/
    
```

6.6. Integrações Externas (se houver)

- APIs de terceiros (pagamentos, mapas, redes sociais)
- Serviços de autenticação (OAuth2, Firebase, etc.)
- Serviços de envio (email, notificações)

6.7. Ambiente e Implantação

- **Ambientes:** desenvolvimento, homologação, produção
- **Contêineres:** Docker/Docker Compose (se usado)
- **CI/CD:** GitHub Actions, GitLab CI, Jenkins
- **Hospedagem:** Heroku, Vercel, AWS, GCP, Azure

8. REFERÊNCIAS BIBLIOGRÁFICAS

SOMMERVILLE, I. **Engenharia de Software**. 11ª Edição. São Paulo: Pearson Addison-Wesley, 2017.

Hospital Israelita Albert Einstein. **Quais os riscos do uso frequente de telas por crianças e adolescentes**. Disponível em: Einstein Saúde. Acesso em: 20 março 2026.

Árvore. **Jogos na educação: como eles auxiliam no aprendizado**. Disponível em: Árvore Blog. Acesso em: 20 março 2026.

G1. Novo estudo aponta prejuízos cognitivos em crianças causados pelo excesso de telas.

Disponível em: G1 Alagoas. Acesso em: 20 março 2026.

G1. Rotina de estudos em casa ajuda no aprendizado das crianças, dizem especialistas.

Disponível em: G1 Paraná. Acesso em: 20 março 2026.